

```

// Pin definitions
const int emgpin1 = A0;

// RMS parameters
const int RMS_WINDOW = 35;
float rms_buffer[RMS_WINDOW];
int rms_index = 0;

// Initialization parameters
const int INIT_PERIOD = 3000; // 3 seconds in milliseconds
const int SAMPLE_DELAY = 10; // 10ms between samples
const int NUM_SAMPLES = INIT_PERIOD / SAMPLE_DELAY;
float VOLTAGE_THRESHOLD = 0.0; // Will be set during initialization

// Threshold timing parameters
const unsigned long TIME_THRESHOLD = 1000; // 1 second in milliseconds
const unsigned long DEACTIVATION_TIME = 500; // 0.5 seconds in milliseconds
unsigned long threshold_start_time = 0;
unsigned long below_threshold_start_time = 0;
bool above_threshold = false;
bool waiting_for_deactivation = false;
int threshold_output = 0;

// ADC to voltage conversion
const float ADC_TO_MV = 5000.0 / 1023.0; // Assuming 5V reference

void setup() {
    Serial.begin(9600);
    while(!Serial);
    // Initialize buffer
    for(int i = 0; i < RMS_WINDOW; i++) {
        rms_buffer[i] = 0;
    }
    // Perform initialization
    initializeThreshold();
    Serial.println("Initialization complete!");
    Serial.print("Threshold set to: ");
    Serial.println(VOLTAGE_THRESHOLD);
    Serial.println("Raw,Filtered,Threshold"); // Header for Serial Plotter
}

```

```

void initializeThreshold() {
    Serial.println("Starting 3-second initialization...");
    Serial.println("Please Stay Still");
    float samples[NUM_SAMPLES];
    float sum = 0;
    // Collect samples for 3 seconds
    for(int i = 0; i < NUM_SAMPLES; i++) {
        float raw = analogRead(emgpin1);
        float filtered = rmsFilter(raw);
        samples[i] = filtered;
        sum += filtered;
        delay(SAMPLE_DELAY);
    }
    // Calculate mean
    float mean = sum / NUM_SAMPLES;
    // Calculate standard deviation
    float sumSquareDiff = 0;
    for(int i = 0; i < NUM_SAMPLES; i++) {
        float diff = samples[i] - mean;
        sumSquareDiff += diff * diff;
    }
    float standardDev = sqrt(sumSquareDiff / NUM_SAMPLES);
    // Set threshold to 3 times (2 standard deviations above mean)
    VOLTAGE_THRESHOLD = 3 * (mean + 2 * standardDev);
}

float rmsFilter(float input) {
    // Add new squared value to buffer
    rms_buffer[rms_index] = input * input;
    rms_index = (rms_index + 1) % RMS_WINDOW;
    // Calculate RMS
    float sum = 0;
    for(int i = 0; i < RMS_WINDOW; i++) {
        sum += rms_buffer[i];
    }
    return sqrt(sum / RMS_WINDOW);
}

void checkThreshold(float value) {
    float voltage = value;
    if (voltage >= VOLTAGE_THRESHOLD) {
        // Reset deactivation timer when going above threshold
    }
}

```

```

    waiting_for_deactivation = false;

    if (!above_threshold) {
        // Just went above threshold, start timing
        threshold_start_time = millis();
        above_threshold = true;
    } else {
        // Check if we've been above threshold for 1 second
        if (millis() - threshold_start_time >= TIME_THRESHOLD) {
            threshold_output = 1;
        }
    }
} else {
    // Below threshold
    if (threshold_output == 1) {
        // If we were activated and just went below threshold
        if (!waiting_for_deactivation) {
            below_threshold_start_time = millis();
            waiting_for_deactivation = true;
        }
        // Check if we've been below threshold for 0.5 seconds
        if (waiting_for_deactivation && (millis() - below_threshold_start_time >=
DEACTIVATION_TIME)) {
            threshold_output = 0;
            above_threshold = false;
            waiting_for_deactivation = false;
        }
    } else {
        // Not activated, reset activation timer
        above_threshold = false;
    }
}
}

void loop() {
    // Read raw EMG value
    float raw_emg = analogRead(emgpin1);
    // Apply RMS filter
    float filtered_emg = rmsFilter(raw_emg);
    // Check threshold conditions
    checkThreshold(filtered_emg);
    // Print raw, filtered values, and threshold output

```

```
// Serial.print(raw_emg);  
// Serial.print(",");  
// Serial.print(filtered_emg);  
// Serial.print(",");  
  Serial.println(threshold_output);  
// Serial.print(",");  
// Serial.println(VOLTAGE_THRESHOLD);  
  delay(10);  
}
```